

US08 — Préparer les données du modèle

Simulation du rendement agricole au Sénégal · dataset préparé pour le modèle

7 697

campagnes dans le dataset

8

variables, toutes
saisissables

1960-2015

années de récolte

0

pluie inventée

Synthèse. Le modèle doit servir à **anticiper** : une ONG ou le ministère décrit une campagne et obtient un rendement estimé. Le dataset ne garde donc que des variables que l'utilisateur peut fournir : culture, région, saison, système de production, superficie, année, pluie et température de la campagne. Il part de la table dbt ML.ML_RENDEMENT_CAMPAGNE (9 894 campagnes). Les trois régions sans station prennent le climat de leur voisine, et les campagnes sans pluie connue sont écartées au lieu d'inventer leur climat : il en reste 7 697. Train 1960-2010 (6 456), test 2011-2015 (1 241).

1. Variable cible et périmètre (US07)

Élément	Définition
Question	Pour une campagne donnée (culture, région, saison, superficie, année, climat), quel rendement attendre ?
Usage	Simulation : l'utilisateur décrit la campagne, le modèle estime le rendement et la production
Type de problème	Régression supervisée
Cible	cible_rendement_t_ha : rendement de la campagne (t/ha)
Périmètre	14 régions, 10 cultures, années de récolte 1960-2015
Métriques	MAE (t/ha), RMSE, R^2 , sur le jeu de test uniquement

Le rendement est préféré à la production : la production dépend surtout de la surface, alors que le rendement isole l'effet de la culture, du lieu et du climat. La production est ensuite recalculée pour l'utilisateur : rendement × superficie.

2. Variables nécessaires au modèle

Une seule règle a guidé le choix : **une variable n'entre dans le modèle que si l'utilisateur peut la fournir au moment de la simulation**. Sinon le modèle serait excellent sur le papier, mais inutilisable.

Variable	Rôle	Fournie par
produit	Chaque culture a son niveau de rendement	l'utilisateur
region	Sols, pratiques, climat moyen	l'utilisateur
saison	Principale (juin–novembre) ou contre-saison (février–juillet)	l'utilisateur
systeme_production	Pluvial, irrigué ou décrue	l'utilisateur
superficie_ha	Surface semée	l'utilisateur
annee	Évolution des pratiques dans le temps	l'utilisateur
pluie_cumul_mm	Premier facteur du rendement en culture pluviale	l'utilisateur, ou relevés, ou normale
tavg_moyenne	Température moyenne de la campagne	l'utilisateur, ou relevés, ou normale

Variable écartée	Pourquoi
production_t	rendement = production / superficie : ce serait donner la réponse
rendement de l'année précédente	Inconnu pour une campagne future (les données s'arrêtent en 2015)
anomalies, mois secs, extrêmes...	Calculés par dbt, impossibles à saisir pour un utilisateur

3. Données agricoles et climatiques sélectionnées

Source	Ingestion	Contenu	Période
PostgreSQL	dlt	Campagnes agricoles	1960–1989
MongoDB	Python	Campagnes agricoles	1990–2005
Kafka	Snowflake Sink Connector	Campagnes agricoles	2006–2015
GalsenAPI	Airbyte	Climat mensuel par station : précipitations, températures	jusqu'à 2025

dbt nettoie et fusionne les trois sources agricoles, puis rattache à chaque campagne les relevés climatiques de sa région entre le semis et la récolte (ML.ML_RENDEMENT_CAMPAGNE). Du climat, on garde deux résumés : le **cumul de pluie** et la **température moyenne** de la campagne.

4. Compatibilité temporelle et géographique

Temporelle

L'agriculture est annuelle, le climat mensuel. La saison fixe les mois : **Principale = juin → novembre** (7 661 campagnes), **Contre-saison = février → juillet** (36 campagnes, toutes à Saint-Louis). La pluie de la campagne est la somme de ces mois, sur l'année de récolte. La température n'est mesurée qu'à partir de 1973.

Géographique

La clé commune est la région. Une région à deux stations reçoit leur moyenne, mois par mois. Fatick, Kaffrine et Sedhiou n'ont pas de station (figure 1).

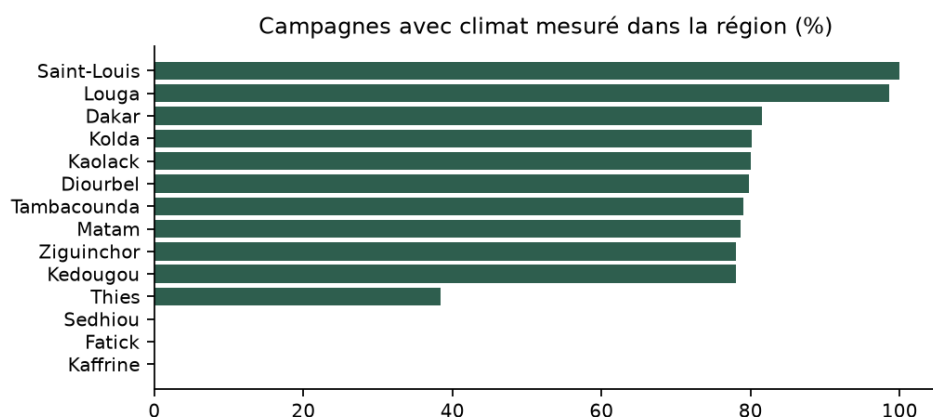


Figure 1 — Part des campagnes avec un climat mesuré dans leur propre région.

5. Construction du dataset combiné

Le script `ml/scripts/donnees.py` part de l'export de la table `dbt` et applique trois règles, dans cet ordre :

- **Système de production vide → Pluvial**, le cas par défaut au Sénégal.
- **Région sans station → climat de la voisine** : Fatick et Kaffrine → Kaolack, Sedhiou → Kolda. C'est une règle métier (proximité géographique), appliquée de la même façon en simulation.
- **Campagne sans pluie connue → écartée**. Plutôt que d'inventer un climat par imputation, on n'apprend que sur des campagnes au climat réellement mesuré (chez elles ou chez la voisine).

Étape	Campagnes
Table dbt ML_RENDEMENT_CAMPAGNE	9 894
dont climat mesuré dans la région	5 760
dont climat de la région voisine	1 937
écartées (pas de pluie connue)	2 197
Dataset final	7 697

6. Traitement du dataset

Valeurs manquantes

Donnée	Traitement	Pourquoi
Pluie de la campagne	campagne écartée (2 197)	La pluie est la variable de simulation : on ne l'invente pas
Température	médiane de train (468 valeurs)	Pas de mesure avant 1973 ; variable secondaire
Système de production	→ Pluvial	Valeur par défaut au Sénégal

La médiane de température est apprise dans le pipeline du modèle, sur les seules données d'entraînement : le jeu de test n'influence pas sa valeur. Aucune valeur manquante n'est remplacée par zéro.

Valeurs aberrantes

Les contrôles `dbt` écartent déjà les valeurs impossibles (mesures négatives, mois invalides). La règle de Tukey ($1,5 \times$ l'écart interquartile, par culture) repère 230 rendements atypiques, surtout pour le riz (102) et le maïs (57). **Ils sont conservés** : ils reflètent la variabilité réelle (riz irrigué, années exceptionnelles) que l'utilisateur doit voir dans les résultats.

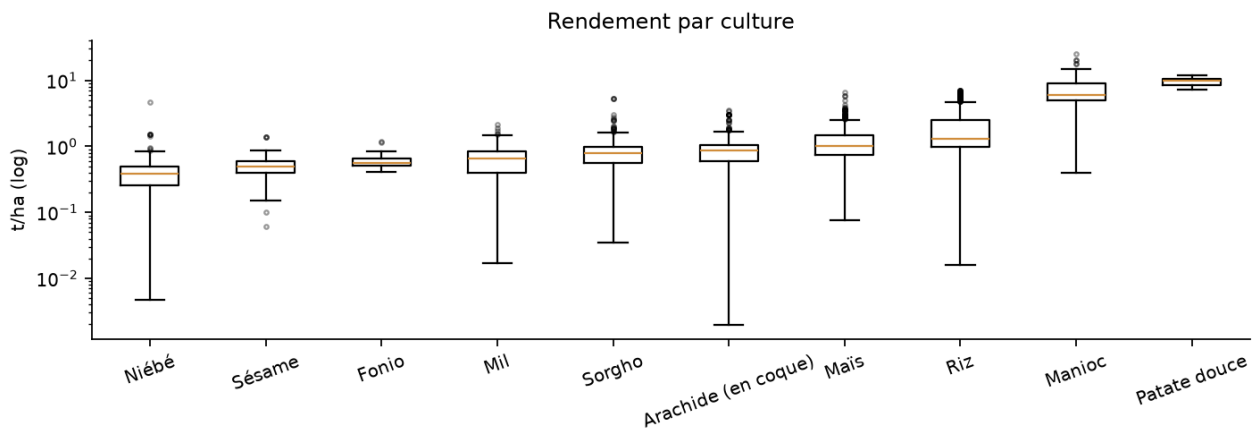


Figure 2 — Rendement par culture (échelle logarithmique) : le manioc et la patate douce rendent dix fois plus que les céréales.

7. Features

Le modèle reçoit les 8 variables de la section 2. Une seule transformation est apprise : les quatre catégories sont encodées en **one-hot** (une colonne par valeur). Une forêt aléatoire n'a besoin ni de standardisation ni de variables construites : elle apprend seule les effets de seuil, par exemple « en dessous de 300 mm, le mil chute ».

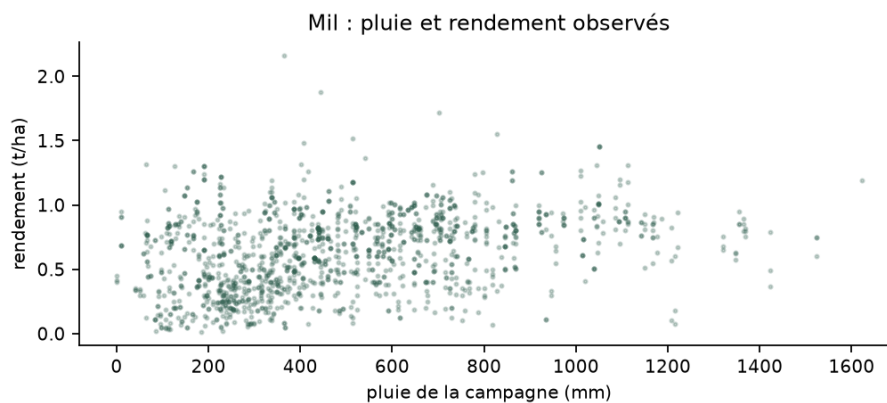


Figure 3 — Mil : la pluie de la campagne et le rendement observé. L'effet est réel mais non linéaire : il est fort sous 400 mm, puis s'atténue.

8. Datasets train et test

Jeu	Années	Campagnes	Part
train	1960-2010	6 456	84 %
test	2011-2015	1 241	16 %

- **Découpage temporel** : on apprend sur le passé et on teste sur les années suivantes, comme en situation réelle.
- Le découpage vient de dbt (colonne jeu) et n'est pas recalculé en Python.
- Le test ne sert jamais à faire un choix (réglage du modèle compris) : seulement à mesurer.

9. Qualité finale du dataset

Contrôle	Résultat
observation_id unique, cible non nulle et positive (dbt)	OK
Pluie connue pour 100 % des campagnes gardées	OK
Valeurs de catégories : 10 cultures, 14 régions, 2 saisons, 3 systèmes	OK
production_t et la cible absentes des variables d'entrée (test)	OK
Règle de la région voisine et campagnes écartées (tests unitaires)	OK
Le jeu de test n'influence pas les réglages (test unitaire)	OK

10. Sauvegarde et versionnement

Élément	Où
Transformations dbt	Git : dbt_project/models
Dataset	Export Parquet daté dans data/processed/ (ml_rendement_campagne_20261007.parquet) ; jamais écrasé
Préparation Python	Git : ml/scripts/donnees.py et config.py
Modèle	ml/models/modele_rendement.pkl (version datée dans le fichier) et metriques.json

```
python -m ml.scripts.export_dataset → data/processed/ml_rendement_campagne_AAAAMMJJ.parquet
```

11. Documentation

Le dictionnaire des variables, les règles de préparation et la façon de relancer le pipeline sont dans ml/README.md. Le notebook 01_exploration.ipynb montre le dataset obtenu ; la construction dbt est décrite dans dbt_project/README_modelisation_ml.md.